



STAGE SGSI

-

CRÉATION D'UN CATALOGUE DES APIs

Documentation technique

Auteur :

KHEIRALLAH Cédric

Superviseur :

MINNE Frédéric

Dernière révision :

27 août 2026

Table des matières

1	Vu d'ensemble du projet	1
2	Diagramme d'architecture	2
3	Structure du projet	3
4	Service backend	4
5	Service frontend	4
6	Service scheduler	4
7	Service proxy	4
8	Limitations	4
8.1	Style du Frontend	4
9	Points d'amélioration	4
10	Conclusion	4

1 Vu d'ensemble du projet

API Catalog est une application web permettant de parcourir les APIs publiées sur une instance WSO2 API Manager, d'afficher leur documentation et de la régénérer automatiquement de manière régulière. Son usage est destiné aux informaticiens du SGSI, des utilisateurs et/ou consultants externes de l'UCLouvain qui souhaitent consulter les APIs publiées sur l'instance API Manager du SGSI afin de faire savoir leur intérêt d'avoir accès à des APIs particulières.

Les utilisateurs du catalogue peuvent parcourir les APIs disponibles sur l'instance, les filtrer, les trier, les consulter et télécharger leur documentation. Cette application offre également aux administrateurs différentes fonctionnalités afin de faciliter sa gestion, notamment :

- Les données peuvent être rafraîchies manuellement
 - Les APIs peuvent être blacklistées afin de les exclure des rafraîchissements de données
 - Le rythme de rafraîchissement automatique peut être configuré par l'administrateur
- et tout cela sans avoir à redéployer l'application.

Le catalogue est déployée en conteneurs via Docker Compose et est constituée de quatre services :

- **backend** : une application FastAPI recevant des requêtes afin d'importer les APIs publiées sur l'API Manager (via l'outil en ligne de commande fourni par WSO2 : APICTL) et de les extraire dans un répertoire de données partagé.
- **frontend** : une application Flask chargée d'afficher le catalogue et la documentation de chaque API à partir du répertoire de données partagé.
- **scheduler** : un lanceur de tâches automatique Supercronic lançant automatique le rafraîchissement automatique en appelant le backend à horaire régulier.
- **proxy** : un proxy Nginx pour le frontend et le backend permettant d'exposer les services en HTTPS.

Dans ce document nous irons plus en détail dans l'architecture, la structure, le fonctionnement et les limitations de l'application. Nous terminerons par des points d'amélioration et une conclusion.

2 Diagramme d'architecture

API Catalog suit la structure suivante :

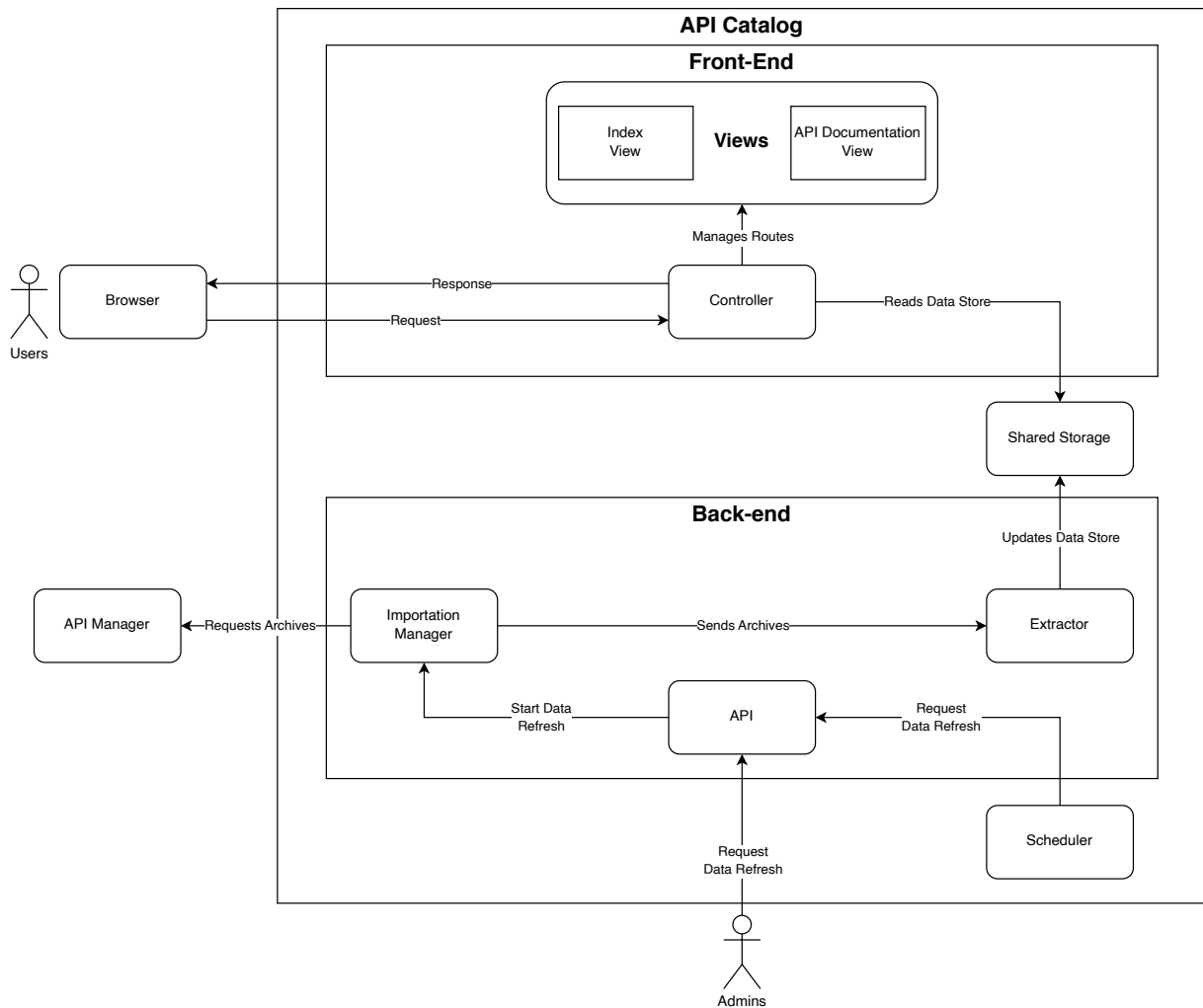


FIGURE 1 – Diagramme d'architecture

Les trois services **backend**, **frontend** et **scheduler** communiquent ensemble via le réseau interne de Docker (**bridge**) et ne sont pas exposés à l'extérieur sauf par l'hôte qui a également accès à ce réseau.

3 Structure du projet

```
1 apictl/  
2     apictl  
3     login.sh  
4     keys.json.example  
5     ...  
6 backend/  
7     api_archive_extractor.py  
8     apictl_client.py  
9     api_manager_client_interface.py  
10    backend.py  
11    blacklist.txt  
12    importation_manager.py  
13    utils.py  
14    ...  
15 data/  
16 docker/  
17     Dockerfile.backend  
18     Dockerfile.frontend  
19     Dockerfile.scheduler  
20 frontend/  
21     static/  
22         font/...  
23         icon/...  
24         redoc/  
25             redoc.standalone.js  
26             ...  
27         filter.js  
28         style.css  
29     templates/  
30         403.html  
31         404.html  
32         415.html  
33         base.html  
34         card.html  
35         contact.html  
36         documentation.html  
37         footer.html  
38         index.html  
39         navbar.html  
40         redoc.html  
41         viewer.html  
42     api_metadata_extractor.py  
43     frontend.py  
44     ...  
45 proxy/  
46     deploy/  
47         certs/...  
48         proxy.conf  
49     snakeoil.sh  
50 scheduler/crontab  
51 compose.yaml  
52 compose.proxy.yaml  
53 .editorconfig  
54 .env.example  
55 ...
```

Listing 1 – Structure du projet

4 Service backend

Le backend FastAPI peut recevoir des requêtes HTTP afin de lancer le processus de rafraîchissement des données à la requête de l'administrateur via l'hôte ou automatiquement depuis le scheduler via le réseau interne de Docker (**bridge**).

5 Service frontend

6 Service scheduler

7 Service proxy

8 Limitations

8.1 Style du Frontend

9 Points d'amélioration

10 Conclusion